

Review



- Last Class:
 - Control hazard
 - Multi-cycle pipelines
- Today's class:
 - Register renaming

Chapter 3: Instruction Level Parallelism (ILP) and its exploitation



- Pipeline CPI = Ideal pipeline CPI + stalls due to hazards
 - invisible to programmer (unlike process level parallelism)
- ILP: overlap execution of unrelated instructions
 - invisible to programmer (unlike process level parallelism)
- Parallelism within a basic block is limited (a branch every 3-6 instructions)
 - Hence, must explore ILP across basic blocks
- The main challenge is overcoming data and control dependencies

Types of dependences



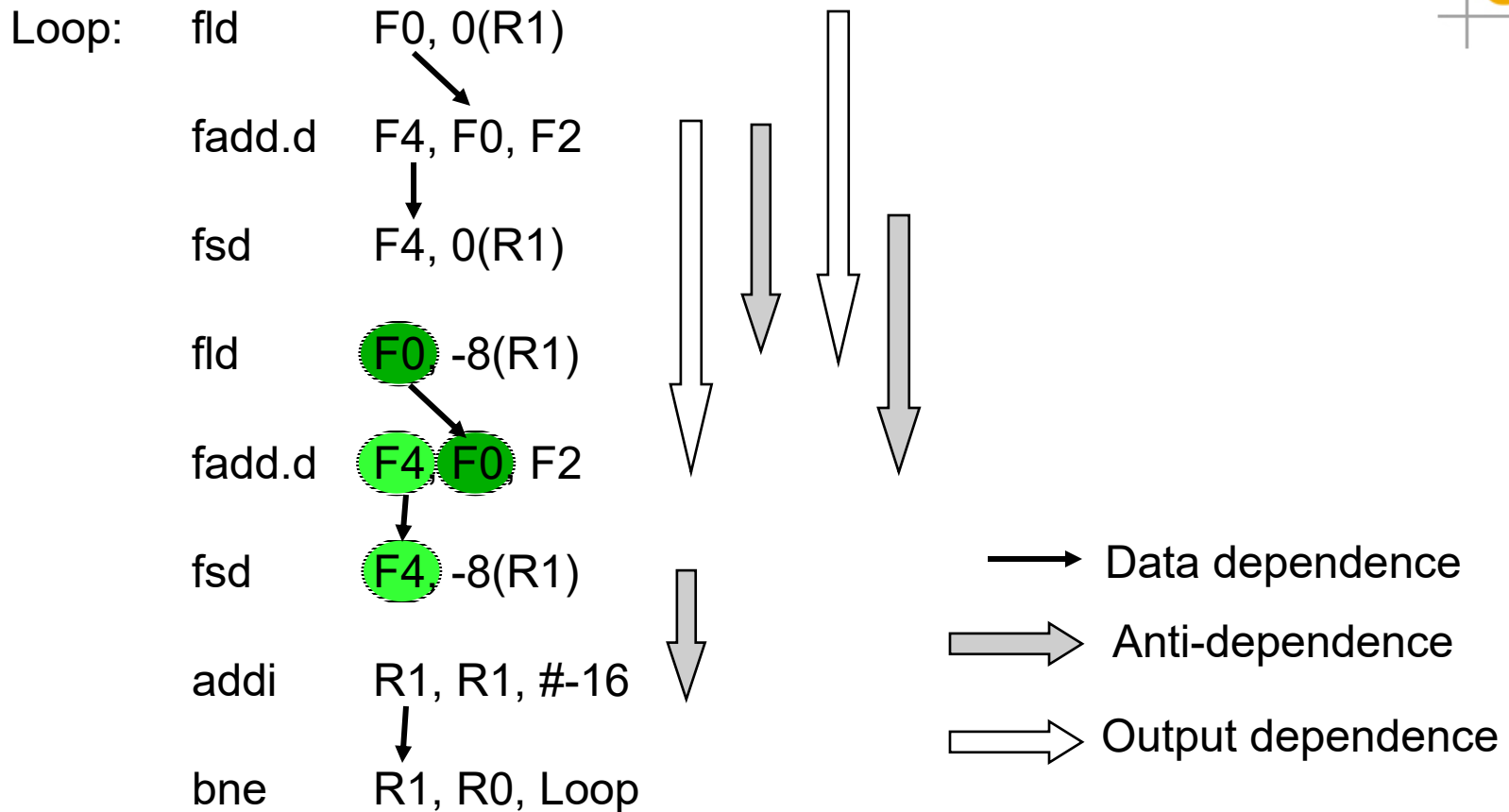
- **True data dependences:** may cause RAW hazards.
 - Instruction Q uses data produced by instruction P or by an instruction which is data dependent on P .
 - dependences are properties of programs, while hazards are properties of architectures.
 - easy to determine for registers but hard to determine for memory locations since addresses are computed dynamically
EX: is $100(R1)$ the same location as $200(R2)$ or even $100(R1)$?
- **False (Name) dependences:** two instructions use the same name but do not exchange data (no data dependency)
 - **Anti-dependence:** Instruction P reads from a register (or memory) followed by instruction Q writing to that register (or memory).
May cause WAR hazards.
 - **Output dependence:** Instructions P and Q write to the same location.
May cause WAW hazards.

Overview of Dependence Analysis

- To what extent can the compiler (or the datapath) reorder instructions? Are there execution-order constraints?

original	possible?	possible?
instr 1 instr 2 consecutive	instr 2 instr 1 consecutive	instr 1 and instr 2 simultaneous

- Instruction dependencies** imply that reordering instructions is not possible
 - true dependence (or, data dep., flow dep.) (cannot reorder)
 $a = .$
 $. = a$ **RAW, read after write**
 - anti-dependence (renaming allows reordering)
 $. = a$
 $a = .$ **WAR, write after read**
 - output dependence (renaming allows reordering)
 $a = .$
 $a = .$ **WAW, write after write**



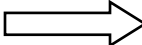
How can you remove name dependences?

Rename the dependent uses of F0 and F4

```

Loop:  fld      F0, 0(R1)
       fadd.d   F4, F0, F2
       fsd      F4, 0(R1)
       fld      F8, -8(R1)
       fadd.d   F9, F8, F2
       fsd      F9, -8(R1)
       addi     R1, R1, #-16
       bne     R1, R0, Loop
  
```



 Data dependence
 Anti-dependence
 Output dependence

Register Renaming



- Can use register renaming to **eliminate** (WAW, WAR) (register) data dependencies – conceptually write each register once
 - + Removes false dependences (WAW and WAR)
 - + Leaves **true** dependences (RAW) intact
- “Architected” vs “Physical” registers
 - Architected (ISA) register names: `$s0, $s1, $s2, $s3, etc`
 - Physical register names: `p1, p2, p3, p4, p5, p6, p7`
- Need two hardware structures to enable renaming
 - A **Map Table** showing the architected register that the physical register is currently “impersonating”
 - A **Free List** of physical registers not currently in use
- When can a physical register be put back on the Free List?

Which Register to Free at Commit ?



- The **over-written** (physical) register can be freed at Commit (i.e., added back to the Free List), so we have to keep track of it during Rename

```
I0: ADD  R2 ← R1 + 1    // reads old R1    -> p5
I1: ADD  R1 ← R3 + R4    // writes R1      -> p12
I2: ADD  R6 ← R1 + R7    // reads new R1
```

- We also need to keep track of the over-written (physical) register so that it can be restored in the Map Table on a recovery from mis-predicted branches and recovery from exceptions

Renaming Example: Initial State

```
lw    $t0, 0($s1)
addu  $t0, $t0, $s2
sw    $t0, 0($s1)
sub    $t0, $s1, $s2
addi   $s1, $s1, -4
bne    $s1, $0, lp
```

\$s1	p1
\$s2	p2
\$t0	p3

Map Table

p4
p5
p6
p7
p8

Free List

Renaming Example: lw Renaming



Over-written Reg

```
lw    $t0, 0($s1) →  
addu  $t0, $t0, $s2  
sw    $t0, 0($s1)  
sub    $t0, $s1, $s2  
addi   $s1, $s1, -4  
bne    $s1, $0, lp
```

\$s1	p1
\$s2	p2
\$t0	p3

Map Table

p4
p5
p6
p7
p8

Free List

Renaming Example: l_w Renaming



Over-written Reg

```
lw    $t0, 0($s1)  →  lw    p4, 0(p1)
addu  $t0, $t0, $s2
sw    $t0, 0($s1)
sub   $t0, $s1, $s2
addi  $s1, $s1, -4
bne   $s1, $0, lp
```

\$s1	p1
\$s2	p2
\$t0	p3

Map Table

p4
p5
p6
p7
p8

Free List

Renaming Example: lw Renaming



Over-written Reg

```
lw    $t0, 0($s1)  →  lw    p4, 0(p1)
addu  $t0, $t0, $s2
sw    $t0, 0($s1)
sub   $t0, $s1, $s2
addi  $s1, $s1, -4
bne   $s1, $0, lp
```

\$s1	p1
\$s2	p2
\$t0	p4

Map Table

p5
p6
p7
p8

Free List

Renaming Example: lw Renaming



Over-written Reg

```
lw    $t0, 0($s1)  →  lw    p4, 0(p1)    [p3]
addu  $t0, $t0, $s2
sw    $t0, 0($s1)
sub   $t0, $s1, $s2
addi  $s1, $s1, -4
bne   $s1, $0, lp
```

\$s1	p1
\$s2	p2
\$t0	p4

p3

Map Table

p5
p6
p7
p8

Free List

Renaming Example: addu Renaming



Over-written Reg

```

lw      $t0, 0($s1)      lw      p4, 0(p1)      [p3]
addu    $t0, $t0, $s2 →  addu    p5, p4, p2
sw      $t0, 0($s1)
sub     $t0, $s1, $s2
addi    $s1, $s1, -4
bne     $s1, $0, lp
    
```

\$s1	p1
\$s2	p2
\$t0	p4

p3

Map Table

p5
p6
p7
p8

Free List

Renaming Example: addu Renaming



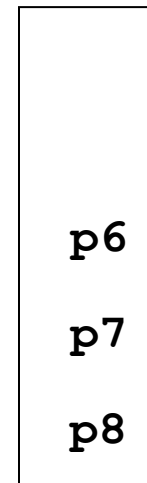
Over-written Reg

lw	\$t0, 0(\$s1)		lw	p4, 0(p1)	[p3]
addu	\$t0, \$t0, \$s2	→	addu	p5, p4, p2	[p4]
sw	\$t0, 0(\$s1)				
sub	\$t0, \$s1, \$s2				
addi	\$s1, \$s1, -4				
bne	\$s1, \$0, lp				

\$s1	p1
\$s2	p2
\$t0	p5

p4 p3

Map Table



Free List

Renaming Example: sw Renaming



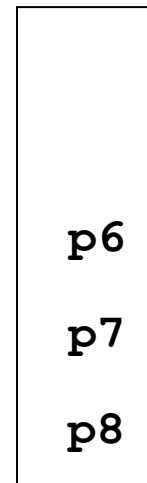
Over-written Reg

lw	\$t0, 0(\$s1)	→	lw	p4, 0(p1)	[p3]
addu	\$t0, \$t0, \$s2		addu	p5, p4, p2	[p4]
sw	\$t0, 0(\$s1)	→	sw	p5, 0(p1)	
sub	\$t0, \$s1, \$s2				
addi	\$s1, \$s1, -4				
bne	\$s1, \$0, lp				

\$s1	p1
\$s2	p2
\$t0	p5

p4 p3

Map Table



Free List

Renaming Example: sub Renaming



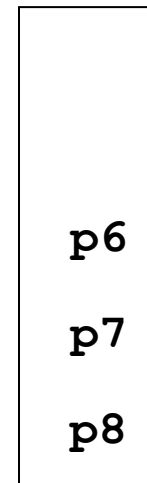
Over-written Reg

lw	\$t0, 0(\$s1)	lw	p4, 0(p1)	[p3]
addu	\$t0, \$t0, \$s2	addu	p5, p4, p2	[p4]
sw	\$t0, 0(\$s1)	sw	p5, 0(p1)	
sub	\$t0, \$s1, \$s2 →	sub	p6, p1, p2	
addi	\$s1, \$s1, -4			
bne	\$s1, \$0, lp			

\$s1	p1
\$s2	p2
\$t0	p5

p4 p3

Map Table



Free List

Renaming Example: sub Renaming

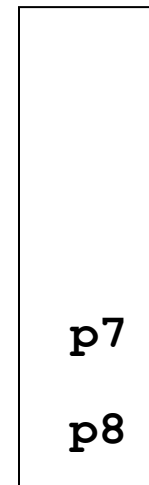


Over-written Reg

lw	\$t0, 0(\$s1)	lw	p4, 0(p1)	[p3]
addu	\$t0, \$t0, \$s2	addu	p5, p4, p2	[p4]
sw	\$t0, 0(\$s1)	sw	p5, 0(p1)	
sub	\$t0, \$s1, \$s2 →	sub	p6, p1, p2	[p5]
addi	\$s1, \$s1, -4			
bne	\$s1, \$0, lp			

\$s1	p1
\$s2	p2
\$t0	p6

p5 p4 p3



Map Table

Free List

Renaming Example: addi Renaming

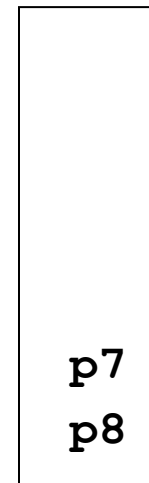


Over-written Reg

lw	\$t0, 0(\$s1)	lw	p4, 0(p1)	[p3]
addu	\$t0, \$t0, \$s2	addu	p5, p4, p2	[p4]
sw	\$t0, 0(\$s1)	sw	p5, 0(p1)	
sub	\$t0, \$s1, \$s2	sub	p6, p1, p2	[p5]
addi	\$s1, \$s1, -4 →	addi	p7, p1, -4	[p1]
bne	\$s1, \$0, lp			

\$s1	p1
\$s2	p2
\$t0	p6

p5 p4 p3



Map Table

Free List

Renaming Example: addi Renaming

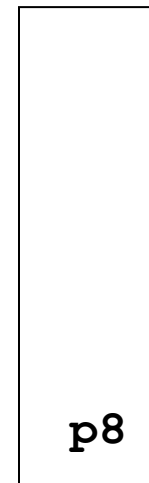


Over-written Reg

lw	\$t0, 0(\$s1)	lw	p4, 0(p1)	[p3]
addu	\$t0, \$t0, \$s2	addu	p5, p4, p2	[p4]
sw	\$t0, 0(\$s1)	sw	p5, 0(p1)	
sub	\$t0, \$s1, \$s2	sub	p6, p1, p2	[p5]
addi	\$s1, \$s1, -4 →	addi	p7, p1, -4	[p1]
bne	\$s1, \$0, lp			

\$s1	p7	p1
\$s2	p2	
\$t0	p6	p5 p4 p3

Map Table



Free List

Renaming Example: bne Renaming

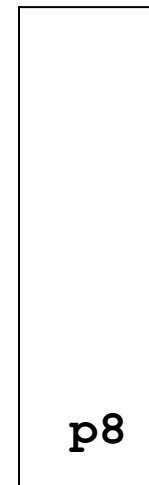


Over-written Reg

lw	\$t0, 0(\$s1)	lw	p4, 0(p1)	[p3]
addu	\$t0, \$t0, \$s2	addu	p5, p4, p2	[p4]
sw	\$t0, 0(\$s1)	sw	p5, 0(p1)	
sub	\$t0, \$s1, \$s2	sub	p6, p1, p2	[p5]
addi	\$s1, \$s1, -4	addi	p7, p1, -4	[p1]
bne	\$s1, \$0, lp	→	bne	p7, p0, lp

\$s1	p7	p1
\$s2	p2	
\$t0	p6	p5 p4 p3

Map Table



Free List

Code Example

RAW

WAR

```
lp(0) : lw    $t0, 0($s1)
        addu   $t0, $t0, $s2
        sw     $t0, 0($s1)
        sub    $t0, $s1, $s2
        addi   $s1, $s1, -4
        bne    $s1, $0, lp
lp(1) : lw    $t0, 0($s1)
        addu   $t0, $t0, $s2
        sw     $t0, 0($s1)
        sub    $t0, $s1, $s2
        addi   $s1, $s1, -4
        bne    $s1, $0, lp
lp(2) : ...
```

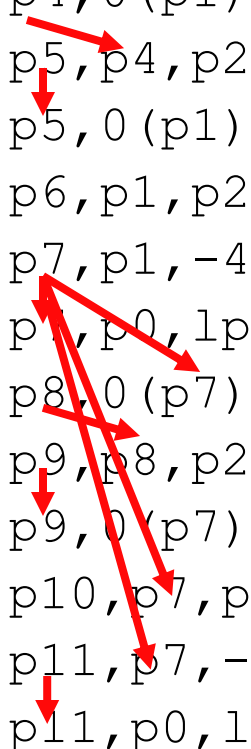
The diagram illustrates data dependencies between instructions in a loop. Blue arrows represent Read-After-Write (RAW) dependencies, green arrows represent Write-After-Read (WAR) dependencies, and red arrows represent Write-After-Write (WAW) dependencies. The instructions are grouped into three iterations: lp(0), lp(1), and lp(2). The first iteration (lp(0)) is followed by a second iteration (lp(1)) that repeats the same sequence of instructions. The diagram shows that the first iteration's instructions are executed first, and then the second iteration's instructions are executed, with dependencies between them. Specifically, the first iteration's 'lw' instruction has a RAW dependency on the second iteration's 'lw' instruction. The first iteration's 'addu' instruction has a RAW dependency on the second iteration's 'addu' instruction. The first iteration's 'sw' instruction has a RAW dependency on the second iteration's 'sw' instruction. The first iteration's 'sub' instruction has a RAW dependency on the second iteration's 'sub' instruction. The first iteration's 'addi' instruction has a RAW dependency on the second iteration's 'addi' instruction. The first iteration's 'bne' instruction has a RAW dependency on the second iteration's 'bne' instruction. The diagram also shows that the first iteration's 'sw' instruction has a WAR dependency on the first iteration's 'lw' instruction. The first iteration's 'sub' instruction has a WAR dependency on the first iteration's 'lw' instruction. The first iteration's 'addi' instruction has a WAR dependency on the first iteration's 'lw' instruction. The first iteration's 'bne' instruction has a WAR dependency on the first iteration's 'lw' instruction. The diagram also shows that the first iteration's 'sw' instruction has a WAW dependency on the first iteration's 'lw' instruction. The first iteration's 'sub' instruction has a WAW dependency on the first iteration's 'lw' instruction. The first iteration's 'addi' instruction has a WAW dependency on the first iteration's 'lw' instruction. The first iteration's 'bne' instruction has a WAW dependency on the first iteration's 'lw' instruction.

Code Example After Renaming

RAW

WAR - none

WAW - none



```
lp(0) : lw      p4, 0 (p1)      #[p3]; cache miss, 3 cycle stall
        addu    p5, p4, p2      #[p4]
        sw      p5, 0 (p1)
        sub     p6, p1, p2      #[p5]
        addi    p7, p1, -4      #[p1]
        bne     p7, p0, lp      #predict taken (and is)
lp(1) : lw      p8, 0 (p7)      #[p6]; cache hit
        addu    p9, p8, p2      #[p8]
        sw      p9, 0 (p7)
        sub     p10, p7, p2     #[p9]
        addi    p11, p7, -4     #[p7]
        bne     p11, p0, lp
lp(2) : ...
```

- As promised, renaming **eliminated** false data dependencies (WAW, WAR) and left true data dependencies (RAW) **intact**